

11- Software testing and maintenance.

Introduction to Testing:

Software testing is a critical element of software quality assurance and represents the ultimate review of specifications design and coding.

Testing objectives:-

According to Glen Myers the testing objectives are,

1. Testing is a process of executing a program with the intent of finding an error
2. A good test case is one that has high probability of finding an undiscovered error.

Testing principles:

Every software engineer must apply following testing principles while performing the software testing,

1. All tests should be traceable to customer requirements
2. Test should be planned long before testing begins
3. Testing should begin "in the small" and progress toward testing "in the large"
4. Exhaustive testing is not possible
5. Testing should be conducted by an independent third party.

Why testing is important?

- Testing is a process that requires more efforts than any other software engineering activity.
- Testing is a set of activities that can be planned in advance and conducted systematically.

Refer example scenarios in Text book (4-3, 4-4, 4-5)

Internal and External View of Testing:-

- The Internal View is also known as white box testing.
- The external View is also known as black box testing.

Black Box Testing:- (Imp (A.U))

- The black box testing is also called as behavioural testing.
- Black box testing methods focus on the functional requirements of the software.
- The black box testing is not an alternative to white box testing and it uncovers different class of errors than white box testing.

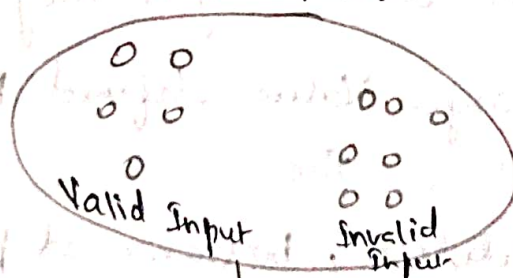
Why to perform black box testing? ②

Black box testing uncovers following types of errors.

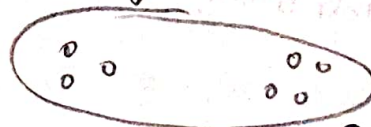
1. Incorrect or missing functions
2. Interface errors
3. Errors in data structures
4. Performance errors
5. Initialization or termination error.

Equivalence partitioning:-

- It is a black box technique that divides the input domain into classes of data.
- From this data test cases can be derived.
- In equivalence partitioning the equivalence classes are evaluated for given input conditions.
- Equivalence class represents a set of valid or invalid states for input conditions.



System



Output generated.

Equivalence class guidelines can be as given below

1. If input conditions specifies a range, one valid and two invalid equivalence classes are defined
2. If an input conditions requires a specific value, one valid and two invalid equivalence classes are defined.
3. If an input conditions specifies a member of a set, one valid and one invalid equivalence class is defined.
- 4) If an input condition is Boolean, one valid and one invalid equivalence class is defined.

For example:-

Area code : Input condition

Boolean - The area code may or may not be present
Input condition, range - Value defined between 200 and 700.

password - Input condition, boolean - A password may or may not be present.

Input condition, value - Seven character string

Command: Input condition, set - containing commands noted before

Boundary Value Analysis (BVA)

(3)

- Boundary value analysis is done to check boundary conditions.
- It is used for developing test cases the boundaries which separate the continuous range of inputs.
- The boundary are identified as part of this technique and then the test cases are written.
- For any scenario, we can write

* Lower boundary cases (using values just below the boundary specified)

* upper boundary cases (using values just above the boundary specified)

* on boundary cases (using the boundary values)

Example

You are going for an adventure park and they have a age restriction like this. These are the valid data that will be arriving on if we apply boundary value analysis.

After your Age [* only 18 to 60 are allowed for this ride]

Boundary Value Analysis

Invalid	Valid	Invalid
(min - 1)	min, min + 1, max, max - 1	max + 1
17	18, 19, 60, 59	61

Example 4.3.4 A program specifies the following for an input field. The program shall accept an input value of a digit integer equal or greater than 2000 and less than or equal to 8000.

i) Equivalence Class partitioning ii) Boundary Value Analysis

sh

Equivalence class partitioning:-

We can have set of test cases which are based on input domain

$$I_1 = \{ \text{Integer} < 2000 \}$$

$$I_2 = \{ \text{Integer} = 2000 \}$$

$$I_3 = \{ \text{Integer} > 2000 \text{ and } < 8000 \}$$

$$I_4 = \{ \text{Integer} = 8000 \}$$

$$I_5 = \{ \text{Integer} > 8000 \}$$

The test cases are

Test case	Integer	Expected output
1	1900	Invalid input
2	2000	Valid input
3	5000	Valid input
4	8000	Valid input
5	9000	Invalid input

ii) Boundary Value Analysis:-

The boundary values can be,

i) less than 2000

ii) Greater than and / or equal to 2000.

- ④
- iii) Less than and less equal to 8000
 - iv) Greater than 8000.

Test case	Integer	Expected output
1	1000	Invalid input
2		
3	2000	Valid input
4	4000	Valid input
5	8000	Valid input
	9000	Invalid input.

White Box Testing:

- * The white box testing is also called as structural testing
- * In white box testing derivation of test cases is according to program structure.
- * To exercise all program statements.

Condition Testing:

- To test the logical conditions in the program module the condition testing is used.
- These conditions can be a Boolean condition or a relational expression.

The condition is incorrect in following situations,

- i) Boolean operator is incorrect, missing or extra
- ii) Boolean variable is incorrect
- iii) Error in relational operator
- iv) Error in arithmetic expression.

* The condition testing focuses on each testing condition in the program.

* The branch testing is a condition testing strategy in which for a compound condition each and every true or false branches are used.

* The domain testing is a testing strategy in which relational expression can be tested using three or four test

Loop Testing:

- Loop testing is a white box testing technique which is used to test the loop constructs.

- Basically there are four types of loops.

1. Simple loop
2. Nested loop
3. Concatenated loop
4. Unstructured loop.

1) Simple loop:

The test can be performed for a number of classes

A.

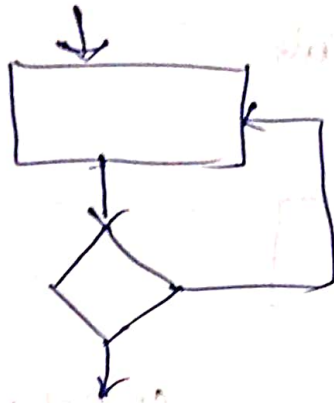
$n=0$, that means skip the loop completely

$n=1$, one pass through the loop is tested

$n=2$, two passes through the loop is tested

$m-m$, Testing is done when there are n passes where $m < n$.

v) perform the testing when number of passes are $n-1$, n , $n+1$ ⑤

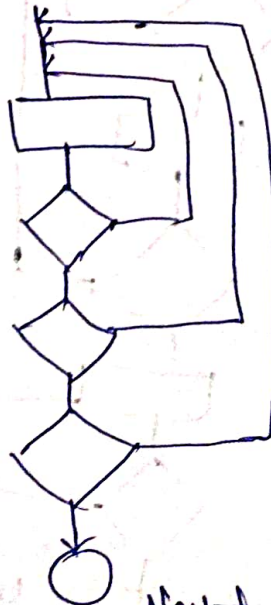


simple loop.

2. Nested loops:-

The nested loop can be tested as follows,

- Testing begins from the innermost loop first. At the same time set all the other loops to minimum values.
- The simple loop test for innermost loop is done
- Conduct the loop testing for the next loop by keeping the outer loop at minimum values.
- This testing process is continued until all the loops have been tested.

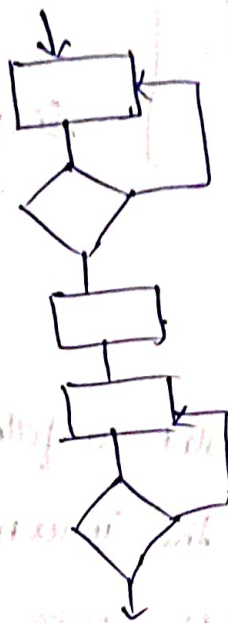


Nested loop.

3. Concatenated loop:

The uncalibrated loops can be tested in the same

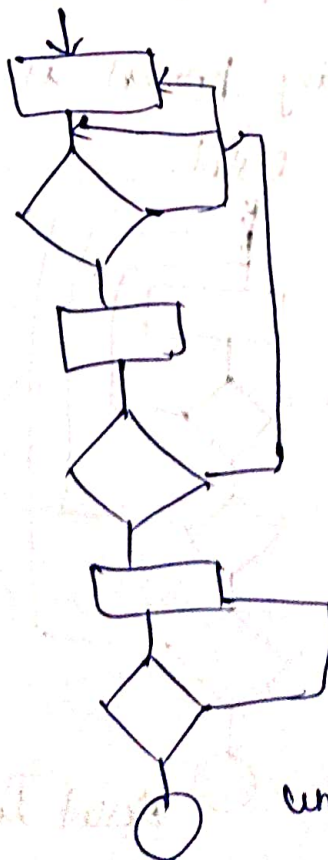
manner as simple loop tests



concatenated loop.

4. unstructured loops:

The testing cannot be effectively conducted for unstructured loops. Hence these types of loops needs to be redesigned.



unstructured loop

Basis Path Testing

It is a structural testing strategy.

step 1: Design flow graph for program.

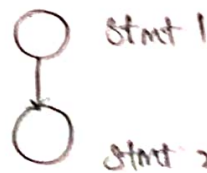
step 2: calculate cyclomatic complexity.

step 3: select basis set path

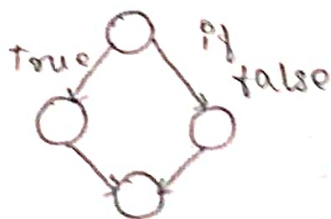
step 4: Generate test cases for paths.

step 1: Design flow graph.

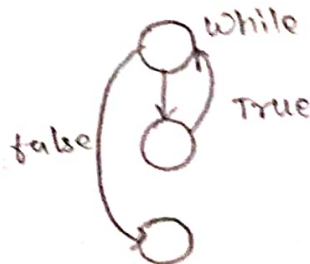
Graph consist of circle called flow graph node.



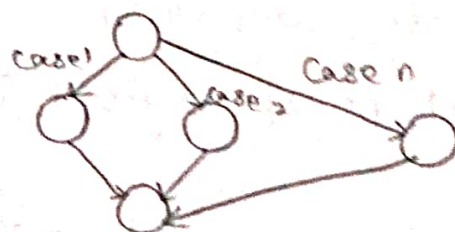
sequence



if-else



while



Case

Ex: void search (int key, int a, int a [])

```

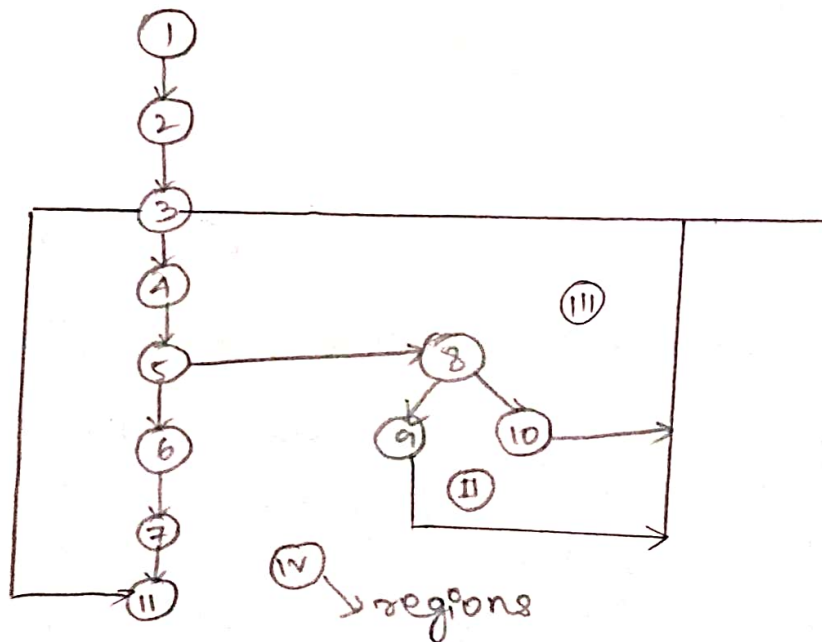
{
    int mid;
    1) int bottom = 0;
    2) int top = n - 1;
    3) while (bottom <= top)
  
```



```

1) { mid = (top + bottom) / 2;
2) if (a[mid] == key) {
3) printf ("Element is present");
4) return; }
5) else {
6) if (a[mid] < key)
7) bottom = mid + 1;
8) else
9) top = mid - 1; } }
10) } //end

```



Step 2: calculate cyclomatic complexity

1) cyclomatic complexity = Total no. of regions in flow graph = 4

2) cyclomatic complexity = $E - N + 2 = 13 \text{ edges} - 11 \text{ nodes} + 2 = 4$

Step 3: Step 2 next select basis set

Path 1: 1, 2, 3, 4, 5, 6, 7, 11

Path 2: 1, 2, 3, 11

(2)

Step 1: Generate test cases for paths.

Test case Id	Test case name	Test case description	Test steps	Test case status (pass/fail)	Test priority	Defect severity
1.			Step Executed Actual			
2.						
3.						

Preconditions:

There should be list of elements arranged in ascending order.

Refer page from (4-19 to 4-34) examples.

Testing strategies:

Begin 'testing-in-the-small' and more 'testing-in-large'

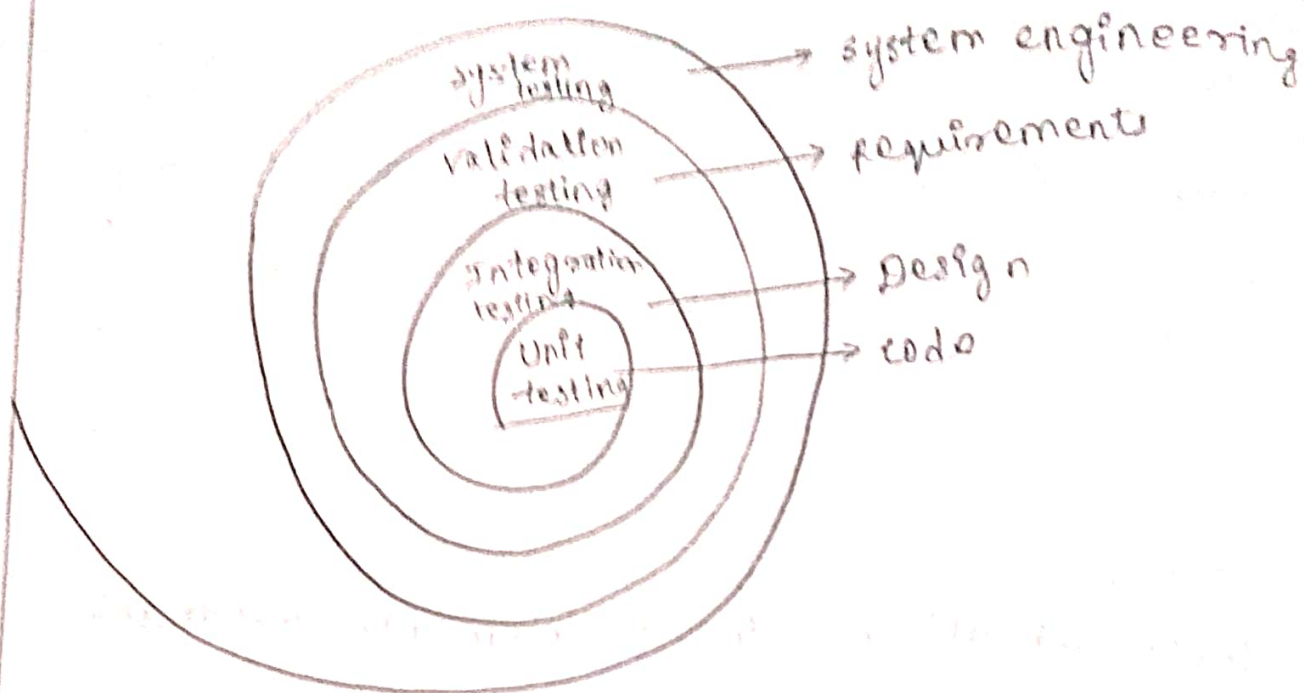
1. unit testing
2. Integration testing
3. validation testing
4. System testing.

1. Unit testing - In this type testing techniques to detect errors.

2. Integration testing - It focus associated with verification, program.

3. validation testing - It provides assurance for software validation, criteria.

4. System testing - All system elements forming system & tested as whole.



S.NO	Black Box testing	White box testing
1.	It is used to test software without knowing internal structure	Used to test software which internal structure is known
2.	Carried out by testers	Carried by software developer
3.	Implementation knowledge is not required	Implementation of knowledge required.
4.	Higher level of testing	Lower level of testing
5.	External testing	Internal testing.

Unit Testing

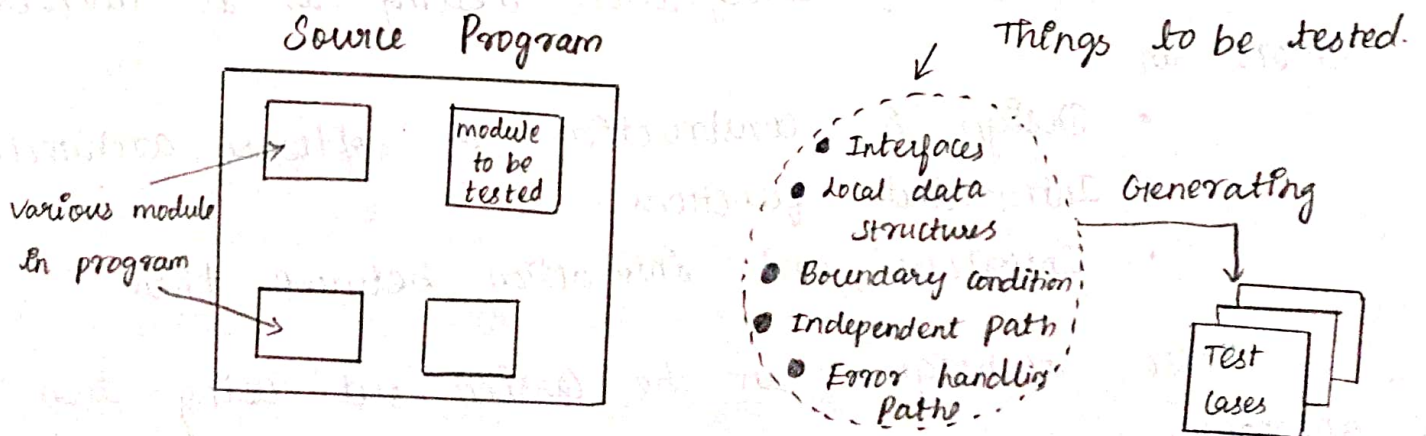
(3)

In unit testing the individual components are tested independently to ensure their quality.

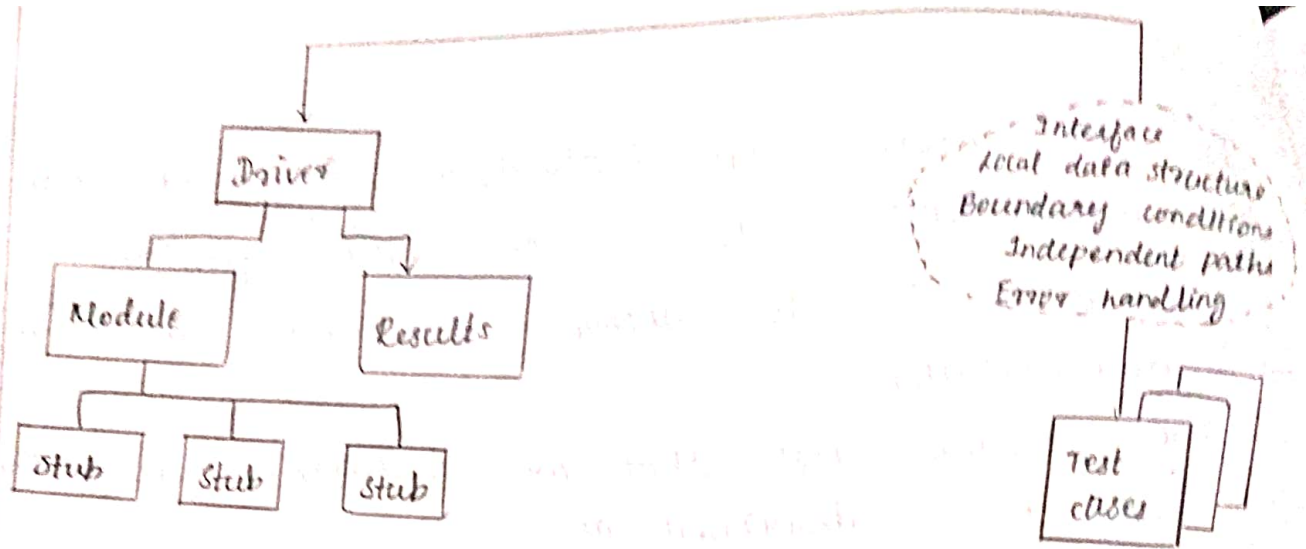
The focus is to uncover the errors in design and manipulation.

The various test that are conducted during the unit test are described as,

1. Module Interfaces are tested for proper information in and out of the program.
2. Local data are examined to ensure that integrity is maintained.
3. Boundary conditions are tested to ensure that the modules operates properly at boundary establishment to limit.
4. All the basic paths are tested for ensuring that all statements in module have been executed only once.
5. All error handling paths should be tested.



6. Drivers and stub software need to be developed to test incomplete software. The "driver" is a program that accepts test data & prints relevant results. "Stub" is a subprogram that used module interfaces.



7. The unit testing is simplified when a component with high cohesion (with one function) is designed.

Integration Testing.

A group of dependent components are tested together to ensure their quality of the integration unit.

The objective is to take unit tested components & build a program structure that has been directed by software design.

The focus of integration testing is to uncover errors in :

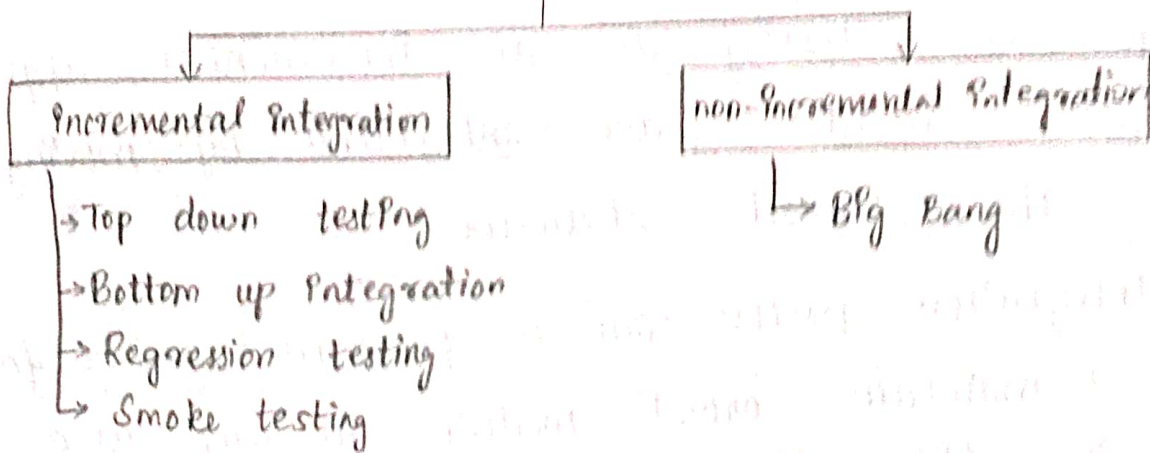
- Design & construction of software architecture.
- Integrated functions
- Interfaces and interaction between them.

These techniques can be carried out using two approaches.

1. The non-Incremental Integration.
2. Incremental Integration.

Integration testing approach

(14)



The non-Incremental Integration is given by big bang approach. All components are combined in advance. The entire program is tested as a whole.

Adv.

* It is simple.

Disadv.

* It is hard to debug

* It is not easy to locate while testing.

* Not easy to validate test results.

An incremental strategy includes.

⇒ Top down Integration.

⇒ Bottom up Integration.

⇒ Regression Testing

⇒ Smoke Testing

The outcome of integration testing is,

⇒ Error in design and construction of software architecture.

⇒ Error from interfaces & interaction between them.

⇒ Errors in resources integration and environment integration.

Top Down Integration Testing

Top down testing is an incremental approach in which modules are integrated by moving down through the control structure.

Integration process can be performed using following steps,

1. maintain control module is used as a test driver & the stubs are substituted for all modules.

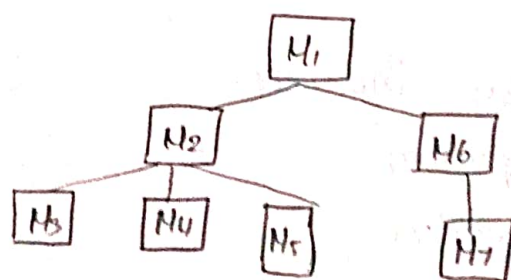
2. Subordinate stubs are replaced one at a time with actual modules.

3. Test are conducted as each module integrated.

4. On completion of each set of test, another stub is replaced with the real module.

5. Regression testing is conducted to prevent the introduction of new errors.

For eg :-



Program structure.

Start Integration from module 1, then we will integrate module M1 first then M2, M3, M4, M5, M6 and M7.

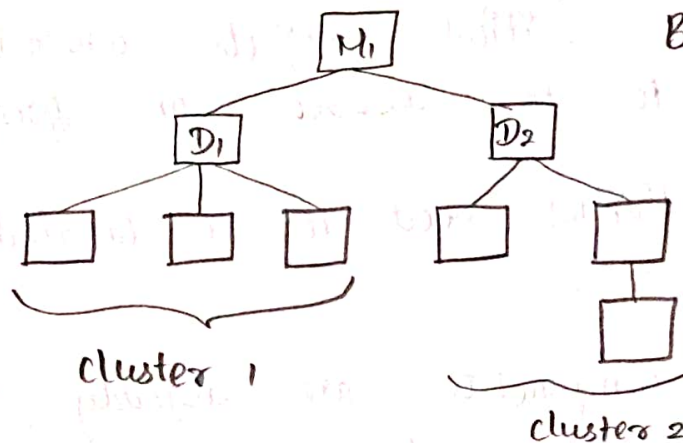
Bottom up Integration Testing

In bottom up Integration the modules at the lowest level are integrated at first, then integration is done by moving upward through the control structure.

The bottom up integration can be carried out ⁽⁵⁾ using following steps.

1. Low-level modules are combined into clusters.
2. A driver program is written to co-ordinate test case input and output.
3. The whole cluster is tested.
4. Drivers are removed & clusters are combined moving upward in the program structure.

For eg :-



Bottom up Integration testing

First components are collected together to form cluster 1 and cluster 2. Then each cluster is tested using a driver program. After testing the driver is removed and clusters are directly interfaced to the modules.

Regression Testing.

It is used to check for defects propagated to other modules by changes made to existing program. Thus regression testing is used to reduce the side effects of the changes.

There are three different classes of test cases involved in regression testing.

system

- Representative sample of existing test cases used exercise all software functions.
- Additional test case focusing software function likely to be affected by the change.
- Test cases that focus on the changed software components.

Smoke Testing.

The Smoke Testing is a kind of Integration testing used for time critical projects where the projects needs to be assessed on frequent basis.

Following activities need to be carried out in Smoke Testing.

1. Software components are already translated into code are integrated into a "build" $\Rightarrow$ data file.
2. A series of test are designed to expose errors from build so that "build" perform its functions correctly.
3. The "build" is integrated with other files.

Benefits of Smoke Testing.

1. Integration risk is minimized.
2. The quality of the end product is improved.
3. Assessment of progress is easy.

system testing

The system test is a series of tests conducted for fully the computer based system

Various types of system tests are

1. Recovery testing
2. Security testing
3. stress testing
4. performance testing

The main focus of such testing is to test

- system functions and performance
- system reliability and recoverability (recovery test)
- system behaviour in the special conditions (stress test)
- system user operations (acceptance test/ alpha test)
- hardware and software integration and collaboration
- integration of external software and the system

Recovery testing.

- Recovery testing is intended to check the system's ability to recover from failures.
- In this type of testing the software is forced to fail and then it is verified whether the system recovers properly or not.

- For automated recovery then reinitialization, checkpoint mechanisms, data recovery and restart are verified.

Security Testing.

- security testing verifies that system protection mechanism prevent improper penetration or data alteration.
- It also verifies that protection mechanisms built into the system prevent intrusion such as unauthorized internal or external access or will fall damage.

Stress Testing.

- determines breakpoint of a system to establish maximum service level
- In stress testing the system is executed in a manner that demands resources in abnormal quantity, frequency or volume
- A variation of stress testing is a technique called sensitivity testing.

Acceptance Testing

(7)

The acceptance testing is a kind of testing conducted to ensure that the software works correctly in the user work environment.

The acceptance testing can be conducted over a period of weeks or months.

Alpha test - The alpha testing is a testing in which the version of complete software is tested by the customer under the supervision of developer.

Advantages

1. users of the system get systematic training to use the new system
2. The acceptance of the system can be immediately known to the developers

disadvantages

1. In depth functionally cannot be tested as software is still under development stage.
2. data used during this testing may not be similar to the business data or actual data.

performance testing.

- performance testing evaluates the run time performance of the software, especially real time software

- In performance testing resource utilization such as cpu load, throughput, response time, memory usage can be measured.

Eg: banking system.

Validation Testing.

- The integrated software is tested based on requirements to ensure that the desired product is obtained

- In validation testing the main focus is to uncover errors in

- software validation can be performed through a series of black box tests.

- After performing the validation tests there exists two conditions.

- finally in validation testing a review is taken to ensure that all the elements of software configuration are developed as per requirements.

Debugging Techniques:

Process of removal of defect. It occurs consequence of testing.

Brute force method - memory dumps, run-time traces examined, program with write statement.

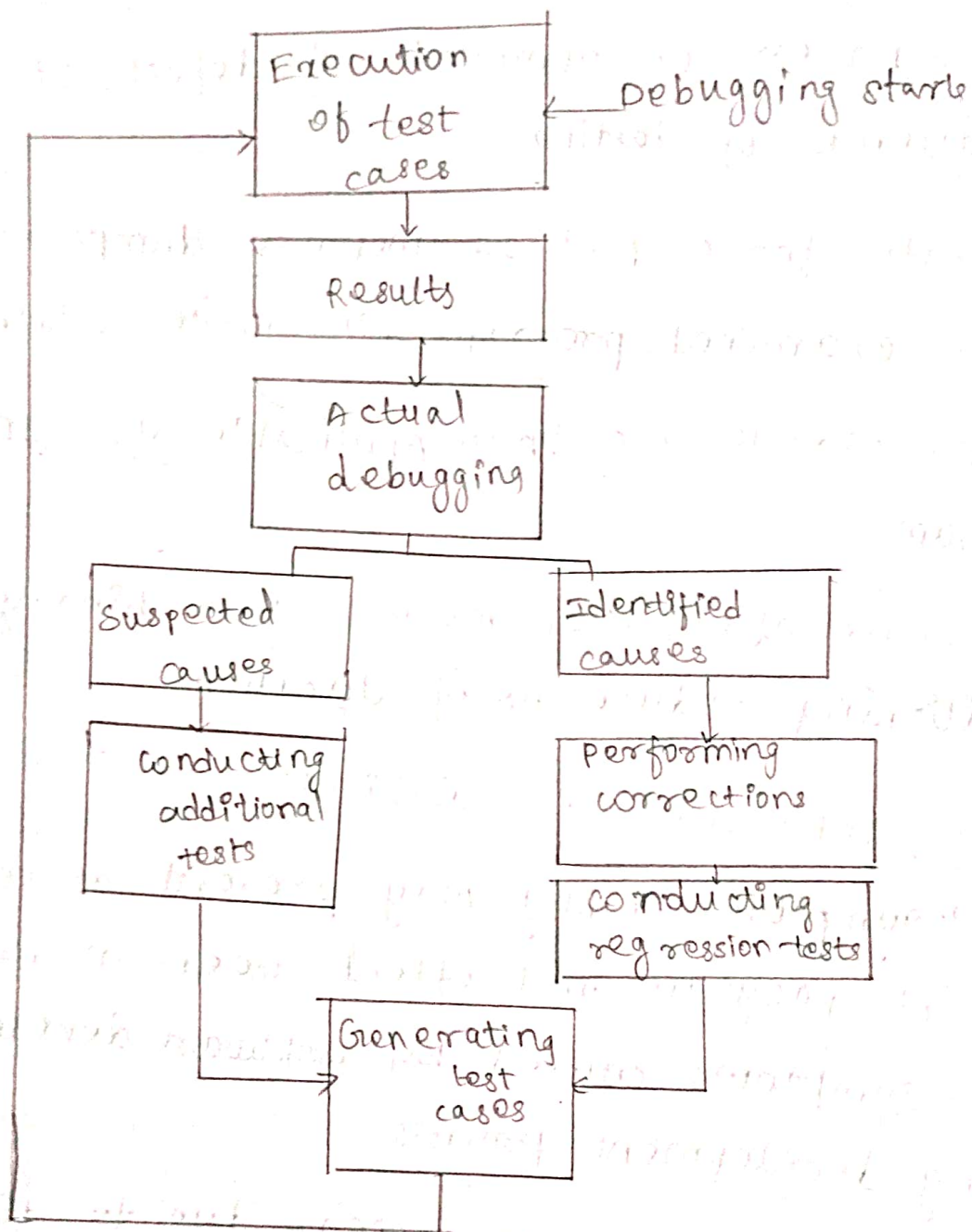
Backtracking method - Applicable for small program.

Cause elimination method - Uses binary partitioning reduce no. of locations.

Why debugging is difficult?

1. Symptoms of bug may present at some part of program and effect seen in module.
2. Symptoms caused by software developer during development process.
3. Symptoms may appear due to timing problems.

A. If developer corrects some error then symptom may disappear.



Debugging technique:

Debugging Process of removal of a defect. It occurs as a consequence of successful testing.

Common approaches in debugging are:

Brute force method: The memory dump and run-time traces are examined and program with write statement is loaded to obtain clues to error causes.

Program Analysis:

There are two ways by which the program analysis is carried out.

1. Static analysis.

2. Dynamic analysis.

1. Static Analysis: Static testing is a technique in which software is tested without executing and program with write statement the code.

Static analysis testing techniques:

Informal reviews: In this technique, the team of the reviews just checks the documents and given comments.

Formal review: It is well structured and document and follows six main steps: planning kick-off, preparation, review meeting, rework follow-up.

Technical review: The team of technical experts will reviews the software for technical specification. It focuses on technique documents such as test strategy, test plan and requirement specification documents.

Walk-through: The author explains the software to the team and teammates can raise questions if they have any.

Inspection process: The meeting is headed by a time trained moderator. A formal review is done, a record is maintained for all the errors and authors are informed to make in feedbacks.

Static code review: code is reviewed without execution, it is checked for syntax, coding standard and code the optimization.

Advantages: It is a fast and easy techniques used to fix error. It helps in identifying flaws in code with the help of automated tools.

Disadvantages: It takes a lot of time to conduct the testing procedure if done manually. Automated tools work for restricted set of programming languages.

2) Dynamic analysis:

Dynamic testing is a process by which code is executed to check how software will perform in a runtime environment.

Advantages: It identifies weak areas in a runtime. It helps in performing detailed analysis of code.

Disadvantages: It is not easy to find a trained tester to perform dynamic testing. It becomes costly to fix errors in dynamic testing.

Symbolic Execution:-

Symbolic execution is a testing technique in which we use symbols to see what is the output

of the code without actually running the code. This helps to find and fix the output of code without actually running the code.

Symbolic execution involves analysing the code without running it. For example: function divide (int a, b)

The result is a symbolic variables we can code's behave in the both case without having specific values for 'a' and 'b' at stage.

```

if (b != 0)
    result = a/b;
else
    result = -1;
return result;
}

```

Model checking:-

Model checking is a technique for verifying that a software system satisfies the desired properties. In this technique, a model of the system is created and then using a model checker we exhaustively explore all possible states of the model to check the properties of the system. For example- consider a ATM system in which we have to design a function for withdrawal of money say withdraw().

This function allows the user to withdraw money from his/her account. In order to implement this function,

The following requirement need to be satisfied

1. The user must have a positive balance in their account in order to withdraw money.

2. The user can't withdraw more money than they have in their account.

3. The user's balance must be updated after each withdrawal.

Before actually executing the code, we can use model checking to verify that the `withdraw()` function meets these requirements.

Model checking is particularly valuable for the complex systems, where manually testing all possible combinations and paths would be the impractical.